Behavioral Clustering of Sports Prediction Markets

Do Kalshi sports markets cluster into distinct behavioral types — and are some of them mispriced?

William Walz & Indigo Lee
Math 364 — Machine Learning in Asset Pricing

1 The Core Idea

Sports prediction markets on Kalshi don't all behave the same way. An NFL game-line contract that resolves in three hours has different price dynamics than an NBA championship futures contract that resolves in seven months. Some markets walk steadily toward truth as information arrives; others stay flat and jump only at resolution; others oscillate noisily without clear directional drift.

We use K-means clustering to group resolved Kalshi sports contracts by their *pre-resolution price dynamics* — features like volatility, drift, time-to-resolution decay, and end-jump size. Then we ask two questions:

Q1 (the main question): Do sports markets cluster into distinct behavioral types? And do those clusters correspond to sport / league / market type, or cut across them in interesting ways?

Q2 (the bonus, calibration check): Are any of these behavioral clusters systematically mispriced? For each cluster, do contracts priced at 70% actually resolve YES 70% of the time?

The pitch: Kalshi organizes sports contracts by sport and league. We argue that contracts cluster more naturally by behavior — and some behavioral clusters reveal pricing inefficiencies that the sport-based organization hides.

2 Tech Stack & Division of Labor

Python (data + analysis + dashboard)

- `requests` for Kalshi REST API
- `pandas` for panel construction and feature engineering
- `scikit-learn` for K-means
- `streamlit` for the data console
- `sqlite3` (stdlib) for local API caching
- Output: cleaned CSVs and a results JSON

R (visualization + presentation graphics)

- `ggplot2` for all plots

- `tidyverse` for any final reshaping
- Indigo's `theme_kalshi()` from his previous project — reuse it directly

Claude Code for the Python pipeline. William writes prompts in his Claude chat, pastes them into Claude Code.

Why split: R's ggplot output is cleaner for slides, and Indigo already has the Kalshi-themed styling infrastructure. Python handles everything upstream where iteration is heavier.

3 The Kalshi API Problem

The Kalshi API has real friction points that need to be addressed before any analysis. Indigo hit several of these in his previous project, including a live-data section that couldn't even render because December 8th data had already aged off the API by compile time.

Specific issues:

- Resolved contracts roll off the API after some weeks — pulls have to happen before the data ages out
- Candle endpoints have time-window limits per request — date ranges have to be chunked
- Ticker discovery requires hitting multiple endpoints (markets, events, series) and reconciling
- Rate limits exist but are not documented well
- Many tickers return zero candles or flat prices, so blind iteration produces noise

Solution: Build a local data console (Section 5, Session 0) that wraps the API in an interactive Streamlit dashboard backed by a SQLite cache. The console is the working environment for the entire project — not a side tool. Once data is cached locally, it's preserved regardless of API state.

4 Data Specification

Universe. Resolved Kalshi sports contracts. Scope determined by what's available — start with whichever sport(s) have the most contracts and cleanest candle data, expand from there. Likely to include NFL, NBA, MLB, and championship/futures contracts across major leagues.

Time range. Past 12–18 months of resolved contracts, depending on how far back the API allows clean retrieval.

Target sample size. 100–250 resolved contracts. Sports contracts are abundant on Kalshi, so we can afford a larger universe than the previous proposal.

Frequency. Hourly or daily candles depending on contract duration. NFL game-line contracts (resolving in hours) need finer granularity; championship futures (resolving in months) work fine on daily.

Filters.

- Contract must have at least 20 non-missing price observations during its lifetime
- Average daily volume ≥ 50 contracts (tune as needed)

- Resolved contracts only — we need known outcomes for both clustering features and calibration
- Exclude contracts where price was effectively flat (degenerate features)

Caching. All API responses go through a local SQLite DB (`data/kalshi_cache.db`) so re-runs are free and the data is preserved against the API ageing it out.

5 Methodology

5.1 Feature engineering

For each resolved contract, compute a small set of behavioral features from its price path. These are the inputs to clustering.

Core features (clustering inputs):

- **Realized volatility:** standard deviation of log-odds changes over the contract's lifetime
- **Directional drift:** average signed change toward the eventual outcome (positive if price walks toward truth, negative if it walks away)
- **Time-to-resolution decay:** fraction of total price movement that occurs in the final 25% of the contract's lifetime (high = late-jumper, low = smooth-mover)
- **End-jump magnitude:** absolute price change in the final 5% of the contract's lifetime
- **Path linearity:** R^2 of a linear regression of price on time — high R^2 means smooth steady drift, low R^2 means oscillation
- **Closing price level:** final price before resolution (controls for whether the contract resolved at high or low confidence)

Metadata (kept for cluster interpretation, not used in clustering):

- Sport (NFL, NBA, MLB, etc.)
- League / market type (game-line, season-long futures, championship, etc.)
- Contract duration (hours to months)
- Resolution outcome (YES = 1, NO = 0)

All features are standardized (z-scored) before clustering.

5.2 K-means clustering

Run K-means on the standardized feature matrix.

Choosing K: Run $K = 2$ through $K = 10$. Use elbow method (within-cluster sum of squares) and silhouette scores. Pick the K with the best silhouette score that's also interpretable.

Outputs:

- Cluster assignments for each contract

- Per-cluster summary: mean feature values, dominant sports/leagues, descriptive label (e.g., "smooth predictors," "late-jumpers," "noise traders")
- Cluster × sport confusion matrix
- Cluster × market-type confusion matrix

5.3 Calibration check (Q2 bonus)

For each cluster, test whether prices match outcomes.

Method: Within each cluster, bin contracts by their closing price into deciles (0–10%, 10–20%, etc.). Compute the empirical resolution rate within each bin. A perfectly calibrated cluster has bin midpoint ≈ empirical YES rate.

Outputs:

- Calibration plot per cluster (predicted vs. realized probability)
- Brier score per cluster: Brier $= \frac{1}{N}\sum(p_i - y_i)^2$ where p_i is closing price and y_i is realized outcome
- Identify any clusters with systematic over- or under-pricing

Caveat: With 100–250 contracts split across K clusters, calibration estimates per cluster will be noisy. We frame this as suggestive evidence, not definitive — but a cluster with strongly biased calibration is worth flagging.

5.4 Robustness checks

- Try hierarchical clustering as a backup; report whichever produces cleaner clusters
- Run clustering on subsamples (e.g., NFL only, NBA only) to check whether clusters are sport-specific or cross-cutting

6 Step-by-Step Execution Plan

Session 0: Data Console (1.5 hours)

Goal: Working Streamlit dashboard wrapping the Kalshi API with SQLite caching.

The console is the project's working environment. Functions written here become the backend for Sessions 1–3.

Four tabs:

- **Contract browser:** Filter by sport, league, status, date range. Sortable table with click-through to price chart.
- **Pull progress:** Live progress for data pulls — tickers attempted, successes, failures, candles per ticker.
- **Panel diagnostics:** Loaded panel summary, missing-data heatmap, contracts flagged as suspicious.

- **Quick cluster preview:** Compute features and run K-means on whatever's loaded; show cluster sizes and silhouette score.

Deliverable: `console.py` (Streamlit dashboard) and `kalshi_lib/` (shared backend).

Session 1: Data Pull (2 hours)

Goal: Clean panel of resolved sports contracts.

Done through the console. Browse, filter, trigger pulls interactively. Final `python/01_pull_data.py` script is exported at the end for reproducibility.

- Use the contract browser to identify target sports tickers
- Run pulls from the dashboard, monitor progress in real time
- Apply filters (liquidity, completeness, resolution status)
- Save raw candles to `data/raw_candles.csv`, resolutions to `data/resolutions.csv`
- All API responses cached to `data/kalshi_cache.db`

Deliverable: Raw candle data + resolution outcomes for 100–250 sports contracts.

Session 2: Feature Engineering (1.5 hours)

Goal: Per-contract feature matrix ready for clustering.

- For each contract, compute the six core features (volatility, drift, decay, end-jump, linearity, closing price)
- Attach metadata (sport, league, market type, duration, outcome)
- Z-score the core features
- Sanity check: distributions of each feature, correlations between features

Deliverable: `data/features.csv` with shape (N contracts $\times$ 6 features + metadata).

Session 3: Clustering (1.5 hours)

Goal: Cluster assignments and cluster-level diagnostics.

- Run K-means for $K = 2$ to 10, save WSS and silhouette scores
- Pick best K, finalize assignments
- Build cluster summary table (mean features per cluster, sport composition, market-type composition)
- Manually label each cluster with a descriptive name
- Confusion matrices: cluster $\times$ sport, cluster $\times$ market type

Deliverable: `results/clusters.csv` with contract $\rightarrow$ cluster mapping, plus diagnostics JSON.

Session 4: Calibration Bonus (1 hour)

Goal: Per-cluster calibration analysis.

- For each cluster, bin contracts by closing price (deciles)
- Compute empirical YES rate per bin
- Compute Brier score per cluster
- Flag clusters with systematic mispricing

Deliverable: `results/calibration.csv` with per-cluster calibration data.

Session 5: Visualization in R (1.5 hours)

Goal: All final plots ready.

Read Python output CSVs into R, make all plots with `theme_kalshi()`:

1. Feature distribution plots (faceted by sport)
2. Elbow + silhouette plots for K selection
3. Cluster scatter (any 2D projection of features, colored by cluster)
4. Cluster $\times$ sport confusion matrix as a tile plot
5. Cluster $\times$ market type confusion matrix as a tile plot
6. Per-cluster calibration plots (predicted vs. realized probability)
7. Bar chart of Brier scores by cluster
8. Example price paths from each cluster (representative contracts)

Deliverable: `figures/` directory with publication-quality plots.

Session 6: Slides + Writeup (1.5 hours)

Goal: Slide deck and short writeup done.

- 10-slide deck (structure below)
- 2-page methodology + results writeup
- Final code cleanup, README
- Optional: rehearse a live demo of the dashboard

Deliverable: `presentation.pdf`, `writeup.pdf`, GitHub repo.

Total: $\sim$10.5 hours over 7 sessions. One session per day, doable in a week.

7 Slide Deck Structure (10 slides)

1. **Title** — "Behavioral Clustering of Sports Prediction Markets" — William Walz & Indigo Lee, Math 364
2. **Motivation** — Sports markets don't all behave the same. Some walk smoothly toward truth, some jump at the end, some oscillate. We cluster them by behavior and check for mispricing.
3. **Data + Console** — Kalshi REST API, ∼N resolved sports contracts, daily/hourly candles. Brief mention of the dashboard. Optional live demo.
4. **Methods** — Six behavioral features per contract → K-means → named clusters. Calibration check per cluster as bonus.
5. **Result 1: Clusters identified** — Show K, silhouette score, and named cluster types with example contracts.
6. **Result 2: Cluster × sport** — Confusion matrix. Highlight cases where behavioral clusters cut across sport boundaries.
7. **Result 3: Cluster × market type** — Same for market type. Tells us whether clusters reflect contract structure or sport.
8. **Result 4: Calibration** — Per-cluster calibration plots. Identify any cluster with systematic mispricing.
9. **Implication** — "Kalshi sports contracts cluster behaviorally in ways that don't match sport/league boundaries. Cluster X shows systematic mispricing."
10. **Limitations & next steps** — Sample size, single platform, calibration noise. Future: out-of-sample test, larger sample, real-time deployment.

8 Repo Structure

```
kalshi-sports-clustering/
|-- CLAUDE.md
|-- README.md
|-- console.py              # Streamlit dashboard
|-- kalshi_lib/             # shared backend code
|   |-- api.py              # API wrapper with SQLite caching
|   |-- features.py         # feature engineering
|   |-- cluster.py          # K-means + diagnostics
|   `-- calibration.py      # calibration analysis
|-- python/                 # final reproducible scripts
|   |-- 01_pull_data.py
|   |-- 02_features.py
|   |-- 03_cluster.py
|   `-- 04_calibration.py
|-- r/
|   `-- plots.R
|-- data/
|   |-- kalshi_cache.db     # SQLite cache
|   |-- raw_candles.csv
```

```
|   |-- resolutions.csv
|   `-- features.csv
|-- results/
|   |-- clusters.csv
|   `-- calibration.csv
|-- figures/
|   `-- *.png
`-- presentation.pdf
```

Workflow rule: Backend logic lives in `kalshi_lib/`. Both `console.py` and the final `python/` scripts call those functions — no logic is written twice.

9 Risks & Mitigations

Risk: Kalshi API rate limits or contract availability gaps.
Mitigation: SQLite cache and Session 0 dashboard exist precisely for this. Pull aggressively early, work from cache afterward.

Risk: Resolved contracts age off the API mid-project.
Mitigation: Pull and cache during Sessions 0–1. Once cached, data is preserved locally.

Risk: Clusters look noisy or non-interpretable.
Mitigation: Try hierarchical clustering as a backup. Reduce feature count if features are redundant. If clusters still look bad, that's itself a finding ("sports markets don't separate cleanly by these features") — frame substantively.

Risk: Calibration estimates are too noisy per cluster.
Mitigation: Frame calibration as suggestive, not definitive. Pool across clusters as a sanity check. If individual cluster calibration is unreliable, report aggregate calibration and per-sport calibration as alternatives.

Risk: One sport dominates the universe and washes out cross-sport patterns.
Mitigation: Stratify the universe selection during Session 1. Use the dashboard to ensure reasonable representation across sports.

Risk: Streamlit dashboard becomes a time sink.
Mitigation: Keep it minimal — four tabs, no styling beyond defaults. Functional visibility, not polish.

10 Roadmap of Future Prompts

When ready to start, William will request prompts in this order from his Claude chat:

1. "Write the API wrapper and SQLite cache layer in `kalshi_lib/api.py`."
2. "Write the Streamlit data console (`console.py`) with the four tabs."
3. "Write the feature engineering code in `kalshi_lib/features.py`."
4. "Write the K-means clustering code in `kalshi_lib/cluster.py` with K-selection diagnostics."

5. "Write the calibration analysis code in `kalshi_lib/calibration.py`."
6. "Write the four reproducible scripts in `python/01_*` through `python/04_*` that wrap `kalshi_lib/` for non-interactive runs."
7. "Write the R plotting script — all plots using `theme_kalshi()`."

Each prompt is generated tailored to what came before so the pipeline composes cleanly.